David Folger & Devin Fromond

CS 3210 | Design of Operating Systems

Lab 04 | Scheduling and Threading

Part One Extra Credit | Least Runtime, Longest Waiter

The policy selected for `SCHED_OTHER` is similar to a *fair* scheduler which ranks tasks by how little CPU they have consumed but gives interactive, I/O-dependent tasks an increased priority by boosting their allocated I/O time.

We define an `other_wait_bonus(…)` method to allow long-waiting tasks the ability to gradually gain higher priority. In our research, we find this method is similar to a lightweight multi-level feedback queue that tracks per-task runtime instead of discrete queues. This method combines historical wait time (`sinfo.wait_time`) in addition to ongoing waits (`last_wait_tick`) then divides by a constant – arbitrarily chosen to be four (or a right shift of two) – to normalize magnitude with respect to other tasks and priorities. Tasks with a higher wait bonus will take priority over other tasks with a similar runtime.

Our scheduler utilizes `other_effective_runtime` to provide an *effective runtime* that considers both the total CPU time the process has been running for (`sinfo.execution_time`) and I/O time (`sinfo.io_time`). This provides a metric that is lower for interactive or I/O-dependent tasks so they are picked before CPU-intensive tasks even if both utilize SCHED_OTHER.

To uphold the nature of least runtime, longest waiter, `proc_is_better` will pick a task with a smaller effective runtime over a task with a longer effective runtime. If the effective runtimes are tied, then we prioritize the one with a longer wait bonus. If this also results in a tie, then we fall back to the RR sequence token and finally PID.

The results of our scheduler are pictured below. The results suggest a performance between RR and `FIFO`, but closer to that of `RR`. We find that `SCHED_OTHER` has a better initial performance but, with many CPU-intensive tasks, the wait-ageing cannot prevent a long tail. We find RR slices the CPU every tick, so no task waits long, thereby mitigating the tail. With `FIFO`, we trivially find one long job blocks everyone, contributing to overall latency – not only on the tail end. More shorter, I/O-dependent tasks may provide better results for `SCHED_OTHER`.